

Universidad Estatal de Sonora
Facultad: Ingenieria
Carrera: Software

Mtro. Francisco Esteban Flores Araiza
Simulación de Sistemas

Actividad 15 · Documento del estudio

Equipo:
Noriega Velazquez Jesus Manuel
Isaac Vivanco Reyna Vanesa
Hernandez Hernandez Carlos Alberto
Sara de los angeles Carmona Catalan

Grupo: 002

Hermosillo, Sonora
Noviembre 2026

Introducción

El presente estudio analiza el flujo de operaciones en la cafetería de la Universidad Estatal de Sonora. El sistema actual opera de manera presencial y secuencial: los clientes realizan una fila para ser atendidos en una de las **2 cajas** disponibles (para pago), y posteriormente pasan a la zona de espera donde **2 cocineros** se encargan de la preparación y entrega de los alimentos. Durante los periodos de receso escolar, la demanda se incrementa de forma masiva, generando cuellos de botella severos que elevan los tiempos de espera de manera crítica.

Problemática y Objetivo

- **Problemática:** Los tiempos de atención promedio de 5 minutos en caja y 15 minutos en cocina se duplican durante las horas pico (llegando a 10 min en caja y 30 min en cocina), lo que satura los recursos disponibles y genera tiempos de ciclo totales inaceptables para los estudiantes.
- **Objetivo:** Simular el comportamiento actual del sistema y evaluar mediante experimentación digital el impacto de introducir una aplicación móvil de pedidos anticipados para descongestionar las filas y reducir los tiempos de espera.
- **Pregunta del estudio:** *¿Qué pasaría si implementamos una app para ordenar y pagar por adelantado?*

Modelo Conceptual (Tabla de Componentes)

Componente	En la cafetería	En SimPy
Entidad	**Cliente(y su pedido)**	**Una función proceso**
Atributos	**“Hora de llegada”, “si usa la app”, “tiempos de atención” y “preparación”**	**llegada, usa_app**
Eventos	**Llega → empieza en caja → termina de pagar → termina de preparar → se entrega**	**request / timeout**
Recursos	**Caja y cocina**	**Resource(env, capacity=n)**
Salidas	**Espera en caja, tiempo total, utilización, largo de fila**	**Lista registro**

Recolección y Análisis de Datos

Las observaciones de campo realizadas directamente en la cafetería permitieron definir las distribuciones estadísticas:

- ➔ **Tiempo entre llegadas:** Distribución Exponencial (tasa λ variable con oleadas en recesos).
- ➔ **Atención en caja:** Distribución Triangular (mín=3, moda=5, máx=10 minutos).
- ➔ **Preparación en cocina:** Distribución Triangular (mín=10, moda=15, máx=30 minutos).

Implementación (Modelo en SimPy)

El sistema fue modelado computacionalmente en Python utilizando la librería SimPy, dividiendo el proceso en un generador de llegadas exponenciales y una función de cliente que gestiona el uso concurrente de los 2 cajeros y 2 cocineros.

Link Colab & Github

- ➔ LINK GOOGLE COLAB
- ➔ LINK DE GITHUB

Validación

Se contrastaron los resultados de las ejecuciones estocásticas frente a los valores empíricos observados en campo. El tiempo total promedio del modelo actual se situó en **58.66 minutos** durante los picos de saturación, lo cual valida de manera congruente los cuellos de botella críticos detectados en los recesos.

Experimentos y Resultados

- **Parámetros de ejecución:** Se configuraron **30 réplicas independientes** por escenario, utilizando una secuencia de semillas estandarizada desde la base **42** hasta la **71**, con un horizonte temporal de **120 minutos de operación continua**.
- **Comparativa de Escenarios:**
  - **Escenario A (Actual - 100% presencial):** Todos los clientes realizan el flujo secuencial por caja y cocina.
  - **Escenario B (Propuesto con App):** Se evaluaron tasas de adopción digital del 30% y 50% de los usuarios.

Escenario A (actual) vs Escenario B(con app)

Promedio de 30 réplicas · semillas 42 a 71 · 120 min por corrida · 2 cajeros y 2 cocineros.

Indicador evaluado	Escenario A (Actual)	Escenario B (30% uso de la app)	Escenario B (50% uso de la app)
Espera promedio en caja	**6.52 min**	**1.46 min**	**0.42 min**
Espera promedio en cocina	**28.21 min**	**33.52 min**	**34.27 min**
Tiempo total en el sistema (ciclo)	**58.66 min**	**56.62 min**	**54.94 min**
Reducción en tiempo total	**línea base (0%)**	**3.5% más rápido**	**6.3% más rápido**

Lectura: la app casi elimina la espera en caja, pero con 2 cocineros la cocina sigue saturada y el tiempo total baja solo 3 a 6 %.

Experimento 2: tiempo total (min) según cocineros y uso de la app

Mismos parámetros que la hoja anterior; solo cambia el número de cocineros.

Cocineros	Sin app	App 30%	App 50%	Reducción vs hoy (50% app)
2	58.7	56.6	54.9	6.5%
4	51.3	49.2	48.5	17.4%
6	45.9	41.4	42.0	28.4%
8	45.0	35.9	34.5	41.2%

Con 8 cocineros y 50 % de app: 34.5 min contra 58.7 min hoy. Sin app, con 8 cocineros la espera pasa a la caja (≈ 21 min).

Análisis de resultados: Con 2 cocineros, introducir la app reduce casi por completo la espera en caja (de 6.52 a 0.42 min con 50 % de uso), pero el tiempo total baja solo entre 3.5 % y 6.3 %, porque la cocina sigue saturada y recibe a todos los clientes. Al aumentar la capacidad de cocina el efecto de la app crece: con 8 cocineros y 50 % de uso el tiempo total baja de 58.7 a 34.5 minutos (41.2 %). Sin app, reforzar solo la cocina traslada la espera a la caja (cerca de 21 min), por lo que ambas medidas son complementarias.

Anexo Código Fuente en Python (SimPy con soporte para App)

Librerías y Parámetros

```
!pip install simpy -q
```

```
import simpy, random, statistics as st
import matplotlib.pyplot as plt

# ===== PARÁMETROS (reemplazar con datos medidos) =====
P = dict(
   inter    = 1.5,          # media entre llegadas (min) -> Exponencial
   caja     = (3, 10, 5),   # Triangular (mín, máx, moda) -> min
   cocina   = (10, 30, 15), # Triangular (mín, máx, moda) -> min
   n_caja   = 2,
   n_cocina= 2,
   horizonte = 120,         # min por corrida
)
N_REPLICAS = 30
SEMILLA_BASE = 42           # semillas 42..71
```

```
# Chequeo de estabilidad: si rho >= 1 la fila crece sin parar
def rho(P):
   m_caja = sum(P['caja'])/3; m_coc = sum(P['cocina'])/3
   lam = 1/P['inter']
   return lam*m_caja/P['n_caja'], lam*m_coc/P['n_cocina']
print("rho caja, rho cocina (escenario A):", [round(x,2) for x in
rho(P)])
```

Modelo

```
def cliente(env, caja, cocina, usa_app, rng, P, res):
   t0 = env.now
   w_caja = 0.0
   if not usa_app:
       with caja.request() as r:
           yield r
           w_caja = env.now - t0
           yield env.timeout(rng.triangular(P['caja'][0], P['caja'][1],
P['caja'][2]))
   t1 = env.now
   with cocina.request() as r:
       yield r
       w_coc = env.now - t1
       yield env.timeout(rng.triangular(P['cocina'][0], P['cocina'][1],
P['cocina'][2]))
   res.append(dict(w_caja=w_caja, w_cocina=w_coc, total=env.now - t0,
app=usa_app))

def generador(env, caja, cocina, p_app, rng, P, res, cnt):
   while True:
       yield env.timeout(rng.expovariate(1.0 / P['inter']))
       cnt['llegaron'] += 1
       env.process(cliente(env, caja, cocina, rng.random() < p_app,
rng, P, res))

def monitor(env, caja, cocina, q, uso, dt=0.1):
   while True:
       uso['caja'] += caja.count*dt; uso['cocina'] += cocina.count*dt
       if abs(env.now - round(env.now)) < 1e-9:      # cada 1 min
           q['caja'].append(len(caja.queue));
q['cocina'].append(len(cocina.queue))
       yield env.timeout(dt)
```

```
def replica(seed, p_app, P):
    rng = random.Random(seed)
    env = simpy.Environment()
    caja = simpy.Resource(env, P['n_caja']); cocina =
simpy.Resource(env, P['n_cocina'])
    res, cnt = [], {'llegaron': 0}
    q = {'caja': [], 'cocina': []}; uso = {'caja': 0.0, 'cocina': 0.0}
    env.process(generador(env, caja, cocina, p_app, rng, P, res, cnt))
    env.process(monitor(env, caja, cocina, q, uso))
    env.run(until=P['horizonte'])
    m = lambda k: st.mean([r[k] for r in res]) if res else float('nan')
    H = P['horizonte']
    return dict(llegaron=cnt['llegaron'], atendidos=len(res),
        w_caja=m('w_caja'), w_cocina=m('w_cocina'), total=m('total'),
        u_caja=uso['caja']/(P['n_caja']*H),
u_cocina=uso['cocina']/(P['n_cocina']*H),
        q_caja=st.mean(q['caja']), q_cocina=st.mean(q['cocina']))

def experimento(p_app, P, n=N_REPLICAS, base=SEMILLA_BASE):
    rs = [replica(base+i, p_app, P) for i in range(n)]
    out = {}
    for k in rs[0]:
        v = [r[k] for r in rs if r[k] == r[k]]
        out[k] = (st.mean(v), 1.96*st.stdev(v)/len(v)**0.5)    # media,
IC95 (+/-)
    return out
```

Escenario A(uso actual): 30 réplicas

```
A = experimento(0.0, P)
for k, (m, ic) in A.items():
    print(f"{k:10s} {m:7.2f}  ± {ic:.2f}")
```

Escenario B: fracción de uso de la app

```
fracs = [0.0, 0.10, 0.30, 0.40, 0.50]
tabla = {f: experimento(f, P) for f in fracs}
print(f"{'app':>5} {'esp.caja':>9} {'esp.cocina':>11} {'total':>7}
{'u.caja':>7} {'u.coc':>6} {'fila caja':>10} {'fila coc':>9}
{'atend/llegaron':>15}")
base_total = tabla[0.0]['total'][0]
for f, o in tabla.items():
    print(f"{f:5.0%} {o['w_caja'][0]:9.2f} {o['w_cocina'][0]:11.2f}
{o['total'][0]:7.2f} "
```

```
        f"{o['u_caja'][0]:7.2f} {o['u_cocina'][0]:6.2f}
{o['q_caja'][0]:10.1f} {o['q_cocina'][0]:9.1f} "
        f"{o['atendidos'][0]:7.0f}/{o['llegaron'][0]:.0f}   reducción:
{100*(1-o['total'][0]/base_total):5.1f}%")
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
x = [f"{int(f*100)}%" for f in fracs]
ax[0].bar(x, [tabla[f]['total'][0] for f in fracs],
yerr=[tabla[f]['total'][1] for f in fracs], color="#8B1A3A")
ax[0].set_title("Tiempo total en el sistema"); ax[0].set_xlabel("%
usuarios de la app"); ax[0].set_ylabel("min")
ax[1].plot(x, [tabla[f]['w_caja'][0] for f in fracs], marker="o",
label="Espera en caja", color="#F5A000")
ax[1].plot(x, [tabla[f]['w_cocina'][0] for f in fracs], marker="o",
label="Espera en cocina", color="#8B1A3A")
ax[1].set_title("Esperas por etapa"); ax[1].set_xlabel("% usuarios de
la app"); ax[1].legend()
plt.tight_layout(); plt.show()
```

Experimento: número de cocineros por uso de la app

```
filas = []
for nc in (2, 4, 6, 8):
   for f in (0.0, 0.30, 0.50):
       o = experimento(f, dict(P, n_cocina=nc))
       filas.append((nc, f, o['w_caja'][0], o['w_cocina'][0],
o['total'][0], o['u_cocina'][0]))
       print(f"cocineros={nc} app={f:.0%} | caja {o['w_caja'][0]:5.1f}
| cocina {o['w_cocina'][0]:5.1f} | total {o['total'][0]:5.1f} |
util.cocina {o['u_cocina'][0]:.2f}")

plt.figure(figsize=(7, 4))
for f, c in zip((0.0, 0.30, 0.50), ("#555", "#F5A000", "#8B1A3A")):
   plt.plot([r[0] for r in filas if r[1] == f], [r[4] for r in filas if
r[1] == f], marker="o", label=f"App {f:.0%}", color=c)
plt.xlabel("Cocineros"); plt.ylabel("Tiempo total (min)");
plt.legend(); plt.title("Tiempo total vs. cocineros"); plt.show()
```

Histogramas de los datos medidos

```
# Las mediciones reales (minutos)
datos = {
   "Tiempo entre llegadas": [0.8, 2.1, 1.3,],
   "Atención en caja":      [4.5, 6.0, 3.8],
   "Preparación en cocina": [14.2, 18.5, 12.0],
```

```
}
fig, axs = plt.subplots(1, 3, figsize=(13, 3.5))
for ax, (nombre, v) in zip(axs, datos.items()):
    if len(v) >= 2:
        ax.hist(v, bins="auto", color="#8B1A3A", edgecolor="white")
        ax.set_title(f"{nombre}\nn={len(v)}  media={st.mean(v):.2f}
mín={min(v):.1f}  máx={max(v):.1f}")
    else:
        ax.set_title(f"{nombre}\n(sin datos)")
plt.tight_layout(); plt.show()
```

Recomendación: Se recomienda a la administración desarrollar la aplicación de pedidos anticipados junto con un refuerzo de la capacidad de cocina. La app sola descongestiona la caja, pero no reduce de forma importante el tiempo total.

Justificación: En los experimentos, la app con 2 cocineros redujo el tiempo total 6.3 %, mientras que la app con 8 cocineros lo redujo 41.2 %. Antes de invertir se sugiere medir cuántos clientes usarían la app.

Reflexiones Individuales

- **Reyna Vanesa Isaac Vivanco:** Se me hizo muy interesante ver cómo con código en Python podemos simular lo que pasa realmente en la cafetería. Al principio batallé un poco con la sintaxis de SimPy, pero ya corriendo las 30 simulaciones con diferentes semillas se ve claramente cómo se saturan los cajeros y la cocina en el receso, igualito a como lo vivimos diario en la escuela.
- **Jesús Manuel Noriega Velázquez:** Trabajar con las 30 corridas automáticas nos ahorró muchísimo tiempo en lugar de hacerlo a mano. Me gustó ver que los números que arroja la computadora coinciden bastante con los tiempos reales que fuimos a medir, lo que nos da confianza de que el modelo quedó bien hecho para la siguiente fase.
- **Carlos Alberto Hernández Hernández:** Esta actividad estuvo buena para entender cómo la programación nos ayuda a resolver problemas de la vida real. Ver que el tiempo de espera en cocina sube a casi media hora en los picos explica perfecto por qué se hacen filas tan pesadas, y con esto ya tenemos la base lista para meter la app.
- **Sara de los Ángeles Carmona Catalán:** Me gustó ver cómo se juntan la teoría y la práctica. Aunque al principio configurar las funciones y los recursos de SimPy parecía complicado, al final pudimos sacar los promedios de las réplicas y comprobar que el sistema actual de la cafetería de verdad sufre de muchos cuellos de botella.

Referencias

- SimPy Documentation. *Introduction to SimPy*. Recuperado de https://simpy.readthedocs.io/
- Kelton, W. D., Sadowski, R. P., & Zupick, D. A. (2015). *Simulation with Arena* (McGraw-Hill Series in Industrial Engineering and Management).
- Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2014). *Discrete-Event System Simulation*. Pearson.